



ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

«ΕΡΓΑΣΙΑ ΣΤΟ ΜΑΘΗΜΑ ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ»

2η εργασία

Γιαννακοπούλου Αριάδνη, 3210028

Μπελεγράτη Μαριάννα, 3210131

Σύρμα Ιωάννα, 3210190

Διδάσκων: Ι. Ανδρουτσόπουλος

Αθήνα

Χειμερινό εξάμηνο 2023-2024

Στόχος της εργασίας είναι η υλοποίηση αλγορίθμων μάθησης ώστε να μπορούν να χρησιμοποιηθούν για την κατάταξη κειμένων σε δύο κατηγορίες (θετική/αρνητική γνώμη).

Μέρος Α

Αρχικά, χρησιμοποιώντας το σύνολο δεδομένων «Large Movie Review Dataset» ή αλλιώς «IMDB dataset», δημιουργήσαμε ένα δικό μας λεξιλόγιο. Στον αλγόριθμο **CreateVoc** πρώτα διαβάζουμε το `imdbvocab` που μας δίνεται από την βάση δεδομένων του IMDB και ύστερα διαβάζουμε όλα τα κείμενα με τις κριτικές(θετικές και αρνητικές) του φακέλου `train`, αποθηκεύοντας τις έγκυρες λέξεις, δηλαδή αυτές που υπάρχουν και στο `imdbvocab`, καθώς και την συχνότητα εμφάνισής τους. Μετά με την χρήση κάποιων υπερπαραμέτρων επιλέγουμε τις χ συχνότερες λέξεις με την προϋπόθεση πως η συχνότητα τους είναι μεγαλύτερη ή και ίση απο 250 αλλά και μικρότερη ή ίση απο 12.000, ώστε να μείνουμε με ένα λεξιλόγιο που θα έχει μέσα τις λέξεις εκείνες που θα μας φανούν πιο χρήσιμες για την διεξαγωγή των πειραμάτων μας. Για την κατάταξη των κειμένων σε δυο κατηγορίες χρησιμοποιήθηκαν οι εξής αλγόριθμοι μηχανικής μάθησης: Naive Bayes, Random Forest και AdaBoost. Ακολουθούν περισσότερες λεπτομέρειες για να τον κάθε ένα ξεχωριστά.

Αφελής ταξινομητής Bayes

Αρχικά διαβάζουμε τα απαραίτητα για την εκτέλεση αρχεία, δηλαδή το λεξιλόγιό που έχουμε ήδη δημιουργήσει και τους φακέλους `train(reviews)` και `test(new_reviews)` αποθηκεύοντας και το αν πρόκειται για θετική(1) ή αρνητική(0) κριτική στις λίστες `labels` και `new_labels` αντίστοιχα.

Προχωράμε στην αξιολόγηση του μοντέλου (`evaluate_model`). Η συνάρτηση αυτή δέχεται όλα τα παραπάνω αρχεία καθώς και ένα `iteration` που καθορίζει τον αριθμό των πειραμάτων που θα πραγματοποιηθούν. Στην περίπτωση μας γίνονται 5 πειράματα εκπαίδευσης. Στο κάθε ένα αυξάνεται το μέγεθος των παραδειγμάτων εκπαίδευσης κατά 5000, ξεκινώντας από 5000 κριτικές και καταλήγοντας στις 25000 (μισές θετικές και μισές αρνητικές). Στην συνάρτηση `evaluate_model` διατηρούμε λίστες στις οποίες αποθηκεύουμε την ορθότητα, την ακρίβεια, την ανάκληση και το F1 για κάθε πείραμα ώστε να εξάγουμε συνολικά συμπεράσματα στο τέλος. Σε κάθε επανάληψη γίνονται οι εξής διαδικασίες: εκπαίδευση του μοντέλου Naive Bayes, παραγωγή προβλέψεων για κάθε κριτική και των δύο δεδομένων(`train-test`), εμφάνιση καμπυλών μάθησης και πινάκων που να δείχνουν το ποσοστό ορθότητας (`accuracy`) στα δεδομένα εκπαίδευσης (όσα έχουν χρησιμοποιηθεί κάθε φορά) και ελέγχου, πινάκων ακρίβειας (`precision`), ανάκλησης (`recall`), F1 καθώς και τις αντίστοιχες καμπύλες τους συναρτήσει του πλήθους των παραδειγμάτων εκπαίδευσης. Αυτό γίνεται με την βοήθεια της `sklearn.metrics` και των συναρτήσεων `f1_score`, `precision_score`, `recall_score`, `accuracy_score`, `classification_report` και της βιβλιοθήκης `matplotlib.pyplot` για την δημιουργία διαγραμμάτων.

Η εκπαίδευση του μοντέλου Naive Bayes γίνεται με την συνάρτηση `train_naive_bayes`, η οποία δέχεται ως παραμέτρους τις κριτικές από τα δεδομένα εκπαίδευσης, τα αντίστοιχα `labels` τους και το `vocabulary` στο οποίο βασίζεται. Αρχικά, δημιουργεί τον πίνακα `X` με την χρήση της συνάρτησης `create_feature_matrix`. Η συνάρτηση αυτή μαζί με την `tokenize` μετατρέπουν κάθε κριτική σε δυαδικό διάνυσμα, όπου κάθε γραμμή αντιπροσωπεύει μια κριτική και κάθε στήλη μία λέξη στο `vocab`, εξασφαλίζοντας την δημιουργία ενός `np.array`, του οποίου τα στοιχεία είναι 0 και 1 ανάλογα με την απουσία ή παρουσία της λέξης στην κριτική. Στην συνέχεια, χωρίζουμε τον `X` σε δύο πίνακες `positive_reviews` και `negative_reviews` με βάση τα `labels` τους. Υπολογίζουμε την πιθανότητα θετικής

και αρνητικής κριτικής(με βάση τον συνολικό αριθμό κριτικών), την συχνότητα εμφάνισης της κάθε λέξης του λεξιλογίου σε κάθε κατηγορία και το σύνολο των θετικών και αρνητικών λέξεων.

Η παραγωγή προβλέψεων με βάση το μοντέλο του Bayes υλοποιείται με την συνάρτηση `predict_naive_bayes`. Πιο συγκεκριμένα, δέχεται μια κριτική(από τα test ή train data ανάλογα τι εξετάζουμε), το λεξιλόγιο καθώς και όσα υπολογίσαμε κατά την εκπαίδευση του μοντέλου. Δημιουργείται το αντίστοιχο δυαδικό διάνυσμα της κριτικής(`review_vector`) και με βάση αυτό υπολογίζεται η θετική και αρνητική πιθανότητα, σύμφωνα με τον τύπο του Bayes. Έπειτα, υπολογίζεται το συνολικό σκορ(πιθανότητα), λαμβάνοντας υπόψη και τα προηγούμενα ποσοστά. Όποιο σκορ είναι υψηλότερο, δηλαδή πιο πιθανό να ισχύει, στέλνεται και ο αντίστοιχος αριθμός 0(negative review) ή 1(positive review).

$$\text{positive_probability} = \frac{P(\text{review_vector}|\text{positive}) \cdot P(\text{positive})}{P(\text{review_vector})}$$

Τα αποτελέσματα των πειραμάτων μας είναι τα εξής:

➤ Πίνακες ποσοστού ορθότητας, ακρίβειας, ανάκλησης, F1 για test και train δεδομένα

```
C:\Windows\System32\cmd.exe X + v
(c) Microsoft Corporation. Με επιφύλαξη κάθε νόμιμου δικαιώματος.

C:\Users\joann\Desktop\ai 2nd>python Bayes.py
start
read voc
read neg pos
read new reviews
Training data size: 5000
Training Classification Report 1 :
      precision    recall  f1-score   support

     0       0.83     0.86     0.85     2500
     1       0.86     0.82     0.84     2500

   accuracy          0.84     0.84     0.84     5000
  macro avg       0.84     0.84     0.84     5000
weighted avg       0.84     0.84     0.84     5000

Test Classification Report 1 :
      precision    recall  f1-score   support

     0       0.80     0.84     0.82    12500
     1       0.83     0.79     0.81    12500

   accuracy          0.81     0.81     0.81    25000
  macro avg       0.81     0.81     0.81    25000
weighted avg       0.81     0.81     0.81    25000

Training data size: 10000
```

```
C:\Windows\System32\cmd.exe X + v

     0       0.80     0.84     0.82    12500
     1       0.83     0.79     0.81    12500

   accuracy          0.81     0.81     0.81    25000
  macro avg       0.81     0.81     0.81    25000
weighted avg       0.81     0.81     0.81    25000

Training data size: 10000
Training Classification Report 2 :
      precision    recall  f1-score   support

     0       0.81     0.85     0.83     5000
     1       0.84     0.80     0.82     5000

   accuracy          0.82     0.82     0.82    10000
  macro avg       0.82     0.82     0.82    10000
weighted avg       0.82     0.82     0.82    10000

Test Classification Report 2 :
      precision    recall  f1-score   support

     0       0.79     0.83     0.81    12500
     1       0.82     0.78     0.80    12500

   accuracy          0.81     0.81     0.81    25000
  macro avg       0.81     0.81     0.81    25000
weighted avg       0.81     0.81     0.81    25000

Training data size: 15000
```

```
C:\Windows\System32\cmd.exe X + v

     0       0.79     0.83     0.81    12500
     1       0.82     0.78     0.80    12500

   accuracy          0.81     0.81     0.81    25000
  macro avg       0.81     0.81     0.81    25000
weighted avg       0.81     0.81     0.81    25000

Training data size: 15000
Training Classification Report 3 :
      precision    recall  f1-score   support

     0       0.77     0.88     0.82     7500
     1       0.86     0.73     0.79     7500

   accuracy          0.81     0.81     0.81    15000
  macro avg       0.81     0.81     0.81    15000
weighted avg       0.81     0.81     0.81    15000

Test Classification Report 3 :
      precision    recall  f1-score   support

     0       0.75     0.89     0.81    12500
     1       0.86     0.70     0.77    12500

   accuracy          0.79     0.79     0.79    25000
  macro avg       0.80     0.79     0.79    25000
weighted avg       0.80     0.79     0.79    25000

Training data size: 20000
```

```
C:\Windows\System32\cmd.exe X + v

     0       0.75     0.89     0.81    12500
     1       0.86     0.70     0.77    12500

   accuracy          0.79     0.79     0.79    25000
  macro avg       0.80     0.79     0.79    25000
weighted avg       0.80     0.79     0.79    25000

Training data size: 20000
Training Classification Report 4 :
      precision    recall  f1-score   support

     0       0.75     0.89     0.82    10000
     1       0.87     0.70     0.78    10000

   accuracy          0.80     0.80     0.80    20000
  macro avg       0.81     0.80     0.80    20000
weighted avg       0.81     0.80     0.80    20000

Test Classification Report 4 :
      precision    recall  f1-score   support

     0       0.74     0.90     0.81    12500
     1       0.87     0.68     0.77    12500

   accuracy          0.79     0.79     0.79    25000
  macro avg       0.80     0.79     0.79    25000
weighted avg       0.80     0.79     0.79    25000

Training data size: 25000
```

```
C:\Windows\System32\cmd.exe X + v - _ X

1      0.87      0.68      0.77      12500

accuracy      0.79      25000
macro avg     0.80      0.79      0.79      25000
weighted avg  0.80      0.79      0.79      25000

Training data size: 25000
Training Classification Report 5 :
precision    recall  f1-score   support

0       0.75    0.89    0.81    12500
1       0.86    0.70    0.77    12500

accuracy      0.81      0.80      0.80      25000
macro avg     0.81      0.80      0.79      25000
weighted avg  0.81      0.80      0.79      25000

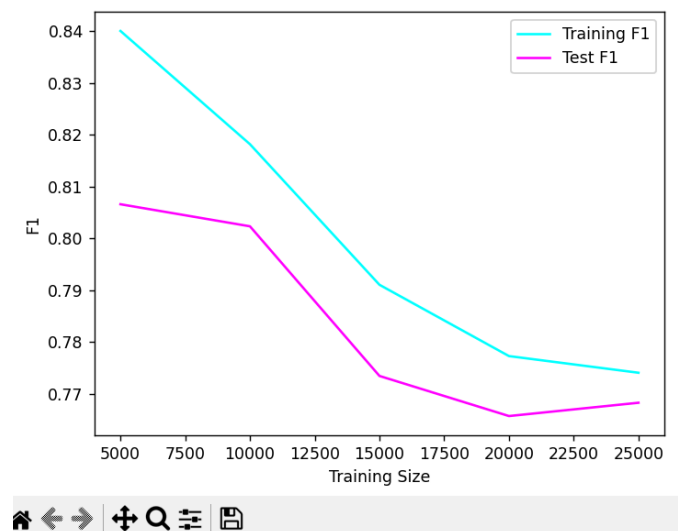
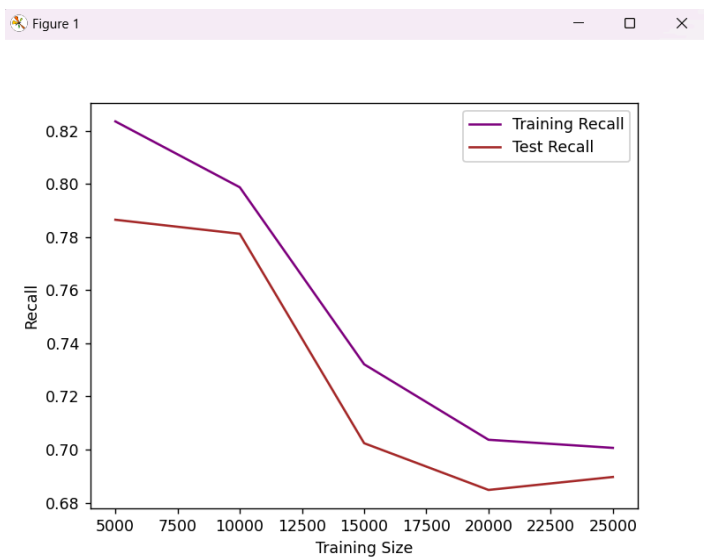
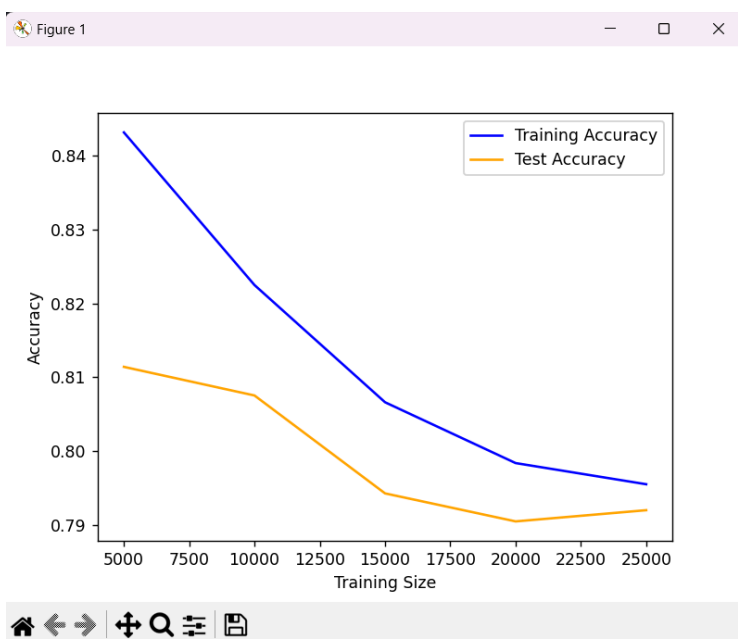
Test Classification Report 5 :
precision    recall  f1-score   support

0       0.74    0.89    0.81    12500
1       0.87    0.69    0.77    12500

accuracy      0.80      0.79      0.79      25000
macro avg     0.80      0.79      0.79      25000
weighted avg  0.80      0.79      0.79      25000

Execution_time: 1166.6068699359894 seconds
C:\Users\joann\Desktop\ai 2nd>
```

➤ Καμπύλες ποσοστού ορθότητας, ακρίβειας, ανάκλησης, F1 για test και train δεδομένα



Random Forest

Η κλάση ID3 υλοποιεί τον αλγόριθμο ID3, δημιουργεί δέντρα απόφασης για προβλήματα ταξινόμησης. Αρχικά, η κλάση `init` δέχεται ένα σύνολο χαρακτηριστικών (features) που θα χρησιμοποιηθούν για τη δημιουργία του δέντρου. Η μέθοδος `fit` είναι υπεύθυνη για τη δημιουργία του δέντρου αποφάσεων. Υπολογίζει τη πιο συχνή κατηγορία (`most_common`) στα δεδομένα εκπαίδευσης και έπειτα καλεί τη μέθοδο `create_tree`, η οποία ξεκινά την αναδρομική δημιουργία του δέντρου. Η μέθοδος `create_tree` δημιουργεί αναδρομικά τους κόμβους του δέντρου με βάση το IG. Εάν τα δεδομένα είναι άδεια, δημιουργείται ένας κόμβος φύλλο που επισημαίνεται με την πιο συχνή κατηγορία. Εάν όλα τα παραδείγματα ανήκουν σε μια μόνο κατηγορία, δημιουργείται ένας κόμβος φύλλο με την αντίστοιχη κατηγορία. Αλλιώς, υπολογίζεται το κέρδος πληροφοριών για όλα τα δυνατά χαρακτηριστικά και επιλέγεται το χαρακτηριστικό με το μεγαλύτερο κέρδος. Μετά δημιουργούνται δύο υποσύνολα δεδομένων, ένα για κάθε τιμή του επιλεγμένου χαρακτηριστικού. Επαναλαμβάνεται η διαδικασία αναδρομικά για κάθε υποσύνολο μέχρι να δημιουργηθεί ένα δέντρο πλήρους βάθους ή να επιτευχθεί μια συνθήκη και να σταματήσει. Η μέθοδος `calculate_ig` υπολογίζει το κέρδος πληροφοριών για ένα χαρακτηριστικό, χρησιμοποιώντας την εντροπία (entropy). Η μέθοδος `predict` δέχεται έναν πίνακα `x`, ο οποίος αναπαριστά τα χαρακτηριστικά των δεδομένων που θέλουμε να προβλέψουμε. Για κάθε δείγμα ξεκινά από τον ριζικό κόμβο και ελέγχει αν ο τρέχον κόμβος είναι φύλλο. Αν είναι, προσθέτει την κατηγορία του φύλλου στις προβλέψεις, αλλιώς κινείται προς το αριστερό ή το δεξί παιδί του κόμβου, ανάλογα με την τιμή του χαρακτηριστικού που ελέγχεται. Το αποτέλεσμα είναι ένας πίνακας με τις προβλέψεις των κατηγοριών για κάθε δείγμα στα δεδομένα εισόδου.

Η κλάση `RandomForest` υλοποιεί έναν απλό ταξινομητή τυχαίων δασών που βασίζεται στον αλγόριθμο ID3. Κατά την αρχικοποίηση, η κλάση δέχεται τον αριθμό των δέντρων που θα δημιουργηθούν (εδώ 100) και το σύνολο των χαρακτηριστικών (features) που θα χρησιμοποιηθούν για κάθε δέντρο (εδώ το `vocab` μας). Η μέθοδος `fit` εκπαιδεύει τα δέντρα. Για κάθε δέντρο, επιλέγει τυχαία ένα υποσύνολο των χαρακτηριστικών (`selected_features`) και εκπαιδεύει ένα δέντρο ID3 με αυτό το υποσύνολο. Η μέθοδος `predict` χρησιμοποιεί τα εκπαιδευμένα δέντρα για να προβλέψει τις κατηγορίες για νέα δεδομένα. Κάθε δέντρο κάνει μια πρόβλεψη, και οι προβλέψεις συγκεντρώνονται με βάση την πλειοψηφία (σε περίπτωση ισοπαλίας, μπορεί να χρησιμοποιηθεί τυχαία επιλογή για το αποτέλεσμα). Τα χαρακτηριστικά που χρησιμοποιούνται για κάθε δέντρο επιλέγονται τυχαία.

Σημείωση: Οι μέθοδοι που δεν αναφέρονται, είναι ίδιες με του Αφελή ταξινομητή Bayes και εξηγούνται παραπάνω

	precision	recall	f1-score	support
0	0.51	0.99	0.67	12500
1	0.77	0.03	0.06	12500
accuracy			0.51	25000
macro avg	0.64	0.51	0.36	25000
weighted avg	0.64	0.51	0.36	25000
	precision	recall	f1-score	support
0	0.50	0.99	0.67	12500
1	0.72	0.03	0.06	12500
accuracy			0.51	25000
macro avg	0.61	0.51	0.36	25000
weighted avg	0.61	0.51	0.36	25000
Execution_time: 78.41119074821472 seconds				

AdaBoost

Αρχικά, ο κατασκευαστής δέχεται ένα όρισμα m , το οποίο αντιπροσωπεύει τον αριθμό των weak learners που θα δημιουργηθούν. Επιπλέον, δημιουργούνται δύο λίστες `self.h` και `self.z` για την αποθήκευση των weak learners και των συντελεστών τους. Η μέθοδος `fit` υλοποιεί την εκπαίδευση των weak learners και υπολογίζει τους συντελεστές. Τα weak learners εκπαιδεύονται να εστιάζουν στα σημεία που οι προηγούμενοι αδύναμοι ταξινομητές (weak learners) απέτυχαν περισσότερο. Η μέθοδος `predict_adaBoost` παίρνει δείγματα και εφαρμόζει το σύνολο των weak learners για να προβλέψει την τελική πρόβλεψη. Οι συντελεστές `self.z` χρησιμοποιούνται για το "βάρος" των αδύναμων ταξινομητών. Τέλος, η μέθοδος `evaluate` χρησιμοποιείται για την αξιολόγηση του μοντέλου AdaBoost στο σύνολο ελέγχου. Εφαρμόζει το μοντέλο σε διάφορα υποσύνολα εκπαίδευσης και συλλέγει τιμές για την απόδοσή του, όπως η ακρίβεια, η ανάκληση, το F1 κλπ. Επίσης, εκτυπώνει αναφορές για κάθε υποσύνολο εκπαίδευσης και ελέγχου.

Σημείωση: Οι μέθοδοι που δεν αναφέρονται, είναι ίδιες με του Αφελή ταξινομητή Bayes και εξηγούνται παραπάνω

Training data size: 5000					
Training Classification Report:					
	precision	recall	f1-score	support	
0	0.82	0.75	0.78	2500	
1	0.77	0.83	0.80	2500	
accuracy			0.79	5000	
macro avg	0.79	0.79	0.79	5000	
weighted avg	0.79	0.79	0.79	5000	
Test Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.73	0.77	12500	
1	0.75	0.83	0.79	12500	
accuracy			0.78	25000	
macro avg	0.78	0.78	0.78	25000	
weighted avg	0.78	0.78	0.78	25000	

Training data size: 10000					
Training Classification Report:					
	precision	recall	f1-score	support	
0	0.82	0.74	0.78	5000	
1	0.76	0.84	0.80	5000	
accuracy			0.79	10000	
macro avg	0.79	0.79	0.79	10000	
weighted avg	0.79	0.79	0.79	10000	
Test Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.73	0.77	12500	
1	0.75	0.83	0.79	12500	
accuracy			0.78	25000	
macro avg	0.78	0.78	0.78	25000	
weighted avg	0.78	0.78	0.78	25000	

Training data size: 15000					
Training Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.75	0.78	7500	
1	0.77	0.83	0.80	7500	
accuracy			0.79	15000	
macro avg	0.79	0.79	0.79	15000	
weighted avg	0.79	0.79	0.79	15000	
Test Classification Report:					
	precision	recall	f1-score	support	
0	0.80	0.74	0.77	12500	
1	0.76	0.82	0.79	12500	
accuracy			0.78	25000	
macro avg	0.78	0.78	0.78	25000	
weighted avg	0.78	0.78	0.78	25000	

Training data size: 20000					
Training Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.75	0.78	10000	
1	0.76	0.83	0.79	10000	
accuracy			0.79	20000	
macro avg	0.79	0.79	0.79	20000	
weighted avg	0.79	0.79	0.79	20000	
Test Classification Report:					
	precision	recall	f1-score	support	
0	0.80	0.74	0.77	12500	
1	0.76	0.82	0.79	12500	
accuracy			0.78	25000	
macro avg	0.78	0.78	0.78	25000	
weighted avg	0.78	0.78	0.78	25000	

Training Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.75	0.78	12500	
1	0.76	0.82	0.79	12500	
accuracy			0.78	25000	
macro avg	0.79	0.78	0.78	25000	
weighted avg	0.79	0.78	0.78	25000	
Test Classification Report:					
	precision	recall	f1-score	support	
0	0.80	0.74	0.77	12500	
1	0.76	0.82	0.79	12500	
accuracy			0.78	25000	
macro avg	0.78	0.78	0.78	25000	
weighted avg	0.78	0.78	0.78	25000	

Figure 1

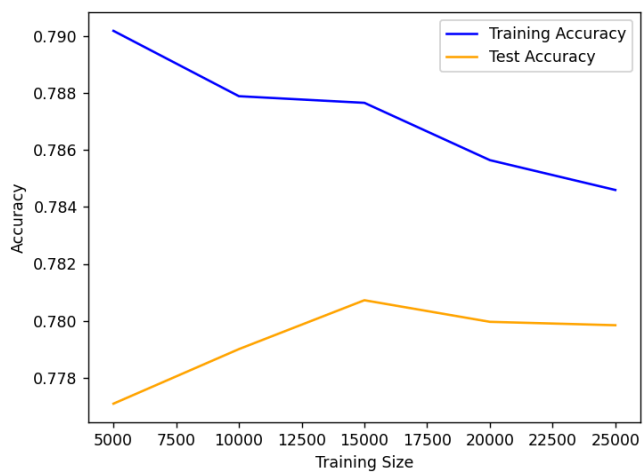


Figure 1

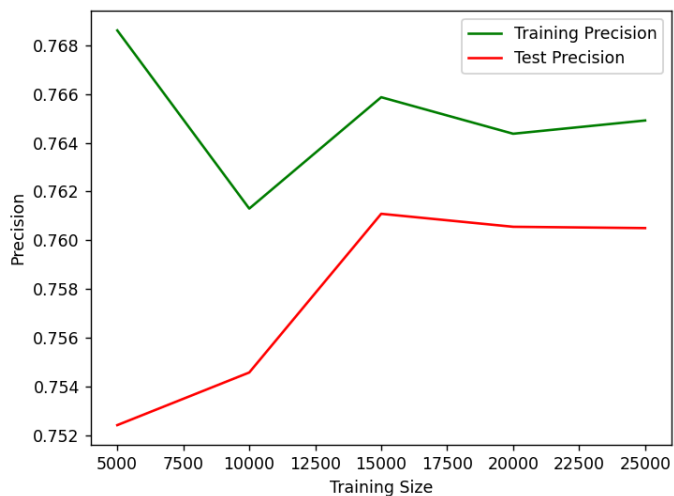


Figure 1

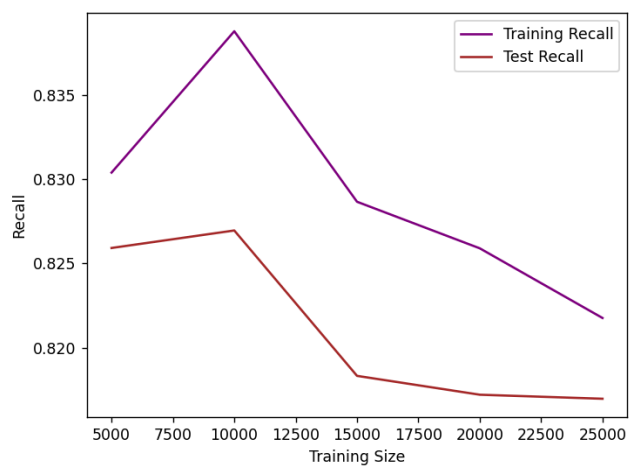
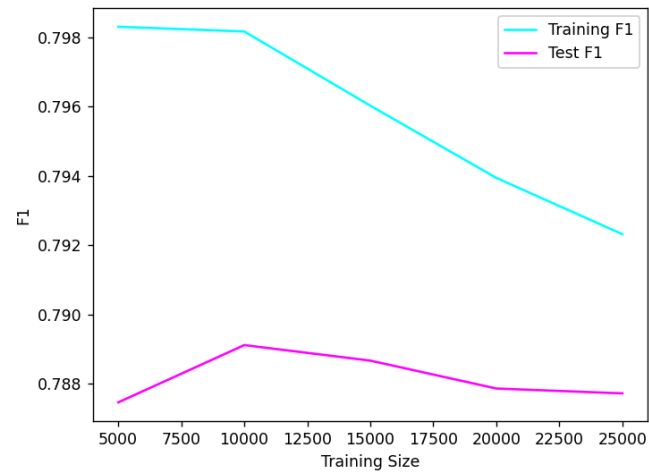


Figure 1



Μέρος Β

Με την χρήση έτοιμων διαθέσιμων υλοποιήσεων αλγορίθμων μηχανικής μάθησης που προσφέρει η Scikit-learn εξαγάγαμε τα παρακάτω αποτελέσματα-συμπεράσματα:

1. Αφελής ταξινομητής Bayes

```
C:\Windows\System32\cmd.e  X  +  v
(c) Microsoft Corporation. Με επιφύλαξη κάθε νόμιμου δικαιώματος.

C:\Users\joann\Desktop\ai 2nd>python bayessklearn.py
start
read voc
read neg pos
read new reviews
Training data size: 5000
Training Classification Report 1 :
      precision    recall  f1-score   support

     0       0.83     0.86     0.84       2500
     1       0.85     0.82     0.84       2500

 accuracy          0.84       0.84       0.84       5000
macro avg          0.84       0.84       0.84       5000
weighted avg       0.84       0.84       0.84       5000

Test Classification Report 1 :
      precision    recall  f1-score   support

     0       0.77     0.83     0.80      12500
     1       0.81     0.75     0.78      12500

 accuracy          0.79       0.79       0.79      25000
macro avg          0.79       0.79       0.79      25000
weighted avg       0.79       0.79       0.79      25000

Training data size: 10000
```

```
C:\Windows\System32\cmd.e  X  +  v

 accuracy          0.79       0.79       0.79      25000
macro avg          0.79       0.79       0.79      25000
weighted avg       0.79       0.79       0.79      25000

Training data size: 10000
Training Classification Report 2 :
      precision    recall  f1-score   support

     0       0.81     0.85     0.83       5000
     1       0.85     0.80     0.82       5000

 accuracy          0.83       0.83       0.83      10000
macro avg          0.83       0.83       0.83      10000
weighted avg       0.83       0.83       0.83      10000

Test Classification Report 2 :
      precision    recall  f1-score   support

     0       0.77     0.84     0.80      12500
     1       0.82     0.75     0.78      12500

 accuracy          0.79       0.79       0.79      25000
macro avg          0.80       0.79       0.79      25000
weighted avg       0.80       0.79       0.79      25000

Training data size: 15000
Training Classification Report 3 :
      precision    recall  f1-score   support
```

```
C:\Windows\System32\cmd.e  X  +  v

     0       0.77     0.84     0.80      12500
     1       0.82     0.75     0.78      12500

 accuracy          0.79       0.79       0.79      25000
macro avg          0.80       0.79       0.79      25000
weighted avg       0.80       0.79       0.79      25000

Training data size: 15000
Training Classification Report 3 :
      precision    recall  f1-score   support

     0       0.82     0.85     0.83       7500
     1       0.84     0.81     0.83       7500

 accuracy          0.83       0.83       0.83      15000
macro avg          0.83       0.83       0.83      15000
weighted avg       0.83       0.83       0.83      15000

Test Classification Report 3 :
      precision    recall  f1-score   support

     0       0.78     0.84     0.81      12500
     1       0.83     0.76     0.79      12500

 accuracy          0.81       0.80       0.80      25000
macro avg          0.81       0.80       0.80      25000
weighted avg       0.81       0.80       0.80      25000

Training data size: 20000
```

```
C:\Windows\System32\cmd.e  X  +  v

     0       0.78     0.84     0.81      12500
     1       0.83     0.76     0.79      12500

 accuracy          0.81       0.80       0.80      25000
macro avg          0.81       0.80       0.80      25000
weighted avg       0.81       0.80       0.80      25000

Training data size: 20000
Training Classification Report 4 :
      precision    recall  f1-score   support

     0       0.82     0.85     0.83      10000
     1       0.84     0.81     0.82      10000

 accuracy          0.83       0.83       0.83      20000
macro avg          0.83       0.83       0.83      20000
weighted avg       0.83       0.83       0.83      20000

Test Classification Report 4 :
      precision    recall  f1-score   support

     0       0.79     0.84     0.81      12500
     1       0.83     0.77     0.80      12500

 accuracy          0.81       0.81       0.81      25000
macro avg          0.81       0.81       0.81      25000
weighted avg       0.81       0.81       0.81      25000

Training data size: 25000
```

```
C:\Windows\System32\cmd.e  X  +  v

     1       0.83     0.77     0.80      12500

 accuracy          0.81       0.81       0.81      25000
macro avg          0.81       0.81       0.81      25000
weighted avg       0.81       0.81       0.81      25000

Training data size: 25000
Training Classification Report 5 :
      precision    recall  f1-score   support

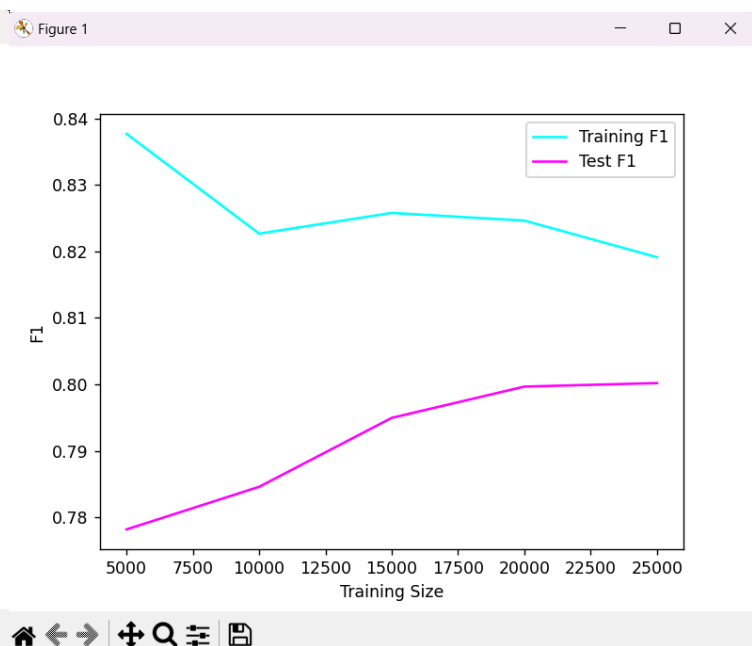
     0       0.81     0.85     0.83      12500
     1       0.84     0.80     0.82      12500

 accuracy          0.82       0.82       0.82      25000
macro avg          0.82       0.82       0.82      25000
weighted avg       0.82       0.82       0.82      25000

Test Classification Report 5 :
      precision    recall  f1-score   support

     0       0.79     0.84     0.81      12500
     1       0.83     0.77     0.80      12500

 accuracy          0.81       0.81       0.81      25000
macro avg          0.81       0.81       0.81      25000
weighted avg       0.81       0.81       0.81      25000
```

Οι βασικές διαφορές που παρατηρούμε σε σχέση με την δική μας υλοποίηση του αλγορίθμου Bayes είναι η πιο ομαλή πορεία των καμπυλών καθώς και η μη κοινή πορεία των δεδομένων εκπαίδευσης και ελέγχου. Στην έτοιμη υλοποίηση όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης: Το ποσοστό ορθότητας(δηλαδή οι σωστές προβλέψεις) αυξάνεται στα test data(από 0.79 στο 0.81) ενώ μειώνεται στα train data(από 0.84 στο 0.82), η ίδια πορεία παρατηρείται και στα υπόλοιπα αποτελέσματα precision(ακρίβεια θετικών προβλέψεων), recall(πραγματικές θετικές περιπτώσεις), F1(συνδυασμός ακρίβειας-ανάκλησης). Στην δική μας υλοποίηση όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης: το accuracy, το recall και το F1 μειώνονται και στις δύο κατηγορίες(accuracy: από 0.84 και 0.81 στο 0.80 και 0.79 στα train και test αντίστοιχα), ενώ το precision έχει ανοδική πορεία στα test data(από 0.83 στο 0.87) και στα train data επιστρέφει περίπου στο ίδιο ποσοστό με το αρχικό κάνοντας μια καθοδική και απότομα ανοδική πορεία(από 0.86 στο 0.84 και τελικά στο 0.865 ξανά).

Random Forest

Training data size: 5000

RANDOM FOREST

Training Classification Report 1 :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2500
1	1.00	1.00	1.00	2500
accuracy			1.00	5000
macro avg	1.00	1.00	1.00	5000
weighted avg	1.00	1.00	1.00	5000

Test Classification Report 1 :

	precision	recall	f1-score	support
0	0.83	0.82	0.82	12500
1	0.82	0.83	0.83	12500
accuracy			0.82	25000
macro avg	0.82	0.82	0.82	25000
weighted avg	0.82	0.82	0.82	25000

Training data size: 10000

RANDOM FOREST

Training Classification Report 2 :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5000
1	1.00	1.00	1.00	5000
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

Test Classification Report 2 :

	precision	recall	f1-score	support
0	0.83	0.83	0.83	12500
1	0.83	0.83	0.83	12500
accuracy			0.83	25000
macro avg	0.83	0.83	0.83	25000
weighted avg	0.83	0.83	0.83	25000

Training data size: 15000

RANDOM FOREST

Training Classification Report 3 :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	7500
1	1.00	1.00	1.00	7500
accuracy			1.00	15000
macro avg	1.00	1.00	1.00	15000
weighted avg	1.00	1.00	1.00	15000

Test Classification Report 3 :

	precision	recall	f1-score	support
0	0.83	0.83	0.83	12500
1	0.83	0.83	0.83	12500
accuracy			0.83	25000
macro avg	0.83	0.83	0.83	25000
weighted avg	0.83	0.83	0.83	25000

Training data size: 20000

RANDOM FOREST

Training Classification Report 4 :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10000
1	1.00	1.00	1.00	10000
accuracy			1.00	20000
macro avg	1.00	1.00	1.00	20000
weighted avg	1.00	1.00	1.00	20000

Test Classification Report 4 :

	precision	recall	f1-score	support
0	0.83	0.84	0.83	12500
1	0.83	0.83	0.83	12500
accuracy			0.83	25000
macro avg	0.83	0.83	0.83	25000
weighted avg	0.83	0.83	0.83	25000

Training data size: 25000

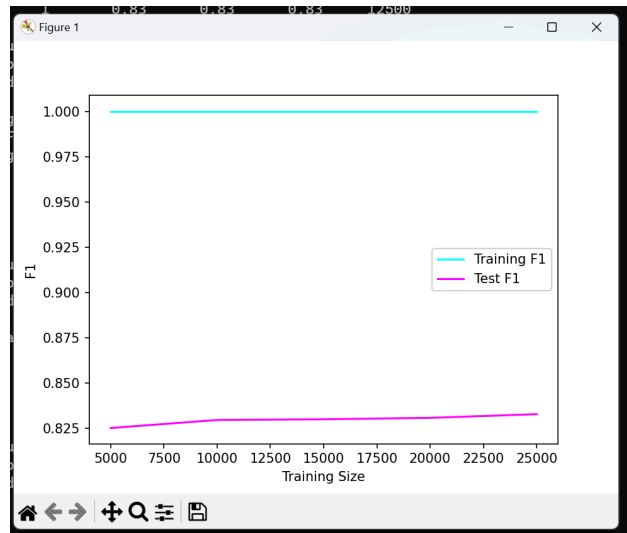
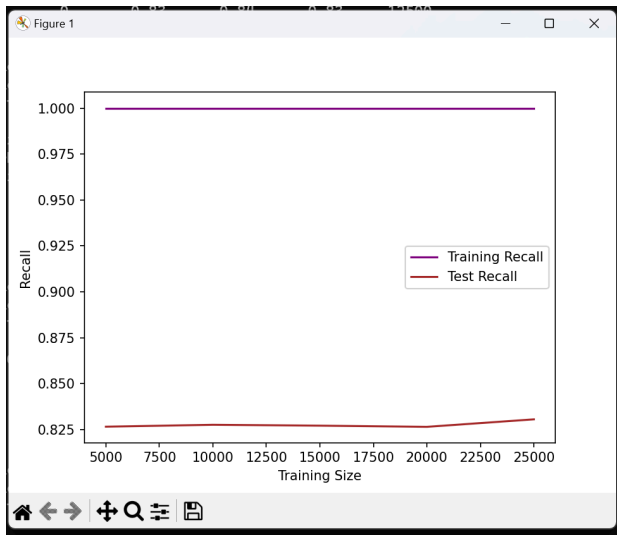
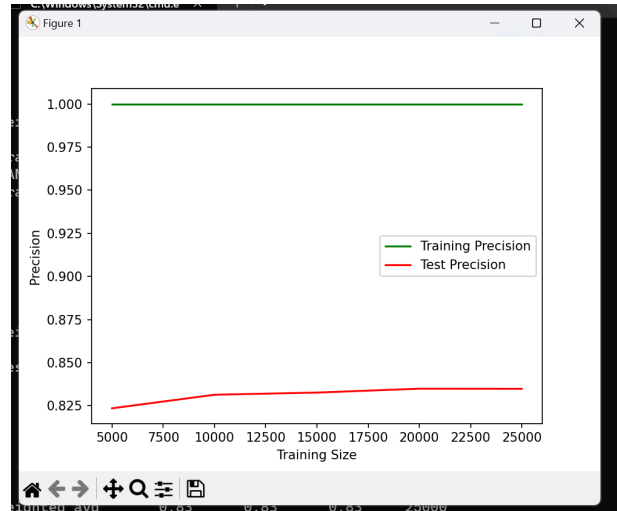
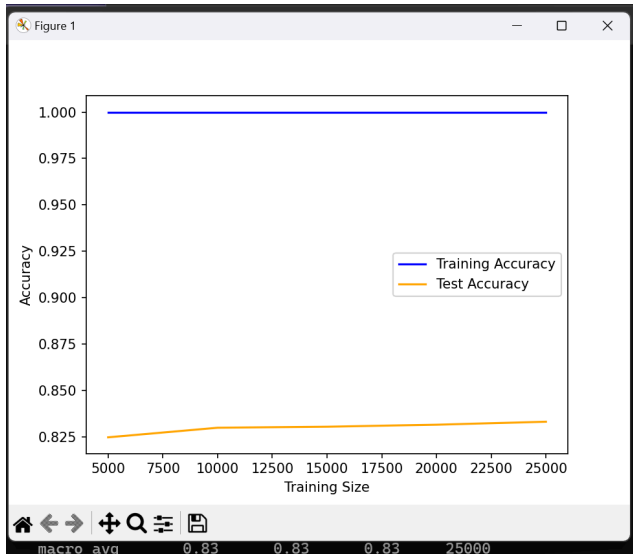
RANDOM FOREST

Training Classification Report 5 :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12500
1	1.00	1.00	1.00	12500
accuracy			1.00	25000
macro avg	1.00	1.00	1.00	25000
weighted avg	1.00	1.00	1.00	25000

Test Classification Report 5 :

	precision	recall	f1-score	support
0	0.83	0.84	0.83	12500
1	0.83	0.83	0.83	12500
accuracy			0.83	25000
macro avg	0.83	0.83	0.83	25000
weighted avg	0.83	0.83	0.83	25000



Η βασική διαφορά που παρατηρούμε σε σχέση με την δική μας υλοποίηση του αλγορίθμου Random Forest είναι η πιο ομαλή πορεία των καμπυλών. Στην έτοιμη υλοποίηση όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης: Το accuracy αυξάνεται στα test data ενώ παραμένει σταθερό στα train data και ίσο με 1.0, η ίδια πορεία παρατηρείται και στα υπόλοιπα αποτελέσματα precision, recall, F1. Στην δική μας υλοποίηση στα train data το accuracy είναι 0.51, το precision 0.64, το recall 0.51 και το f1 0.36. Στα test data το accuracy είναι 0.51, το precision 0.61, το recall 0.51 και το f1 0.36.

AdaBoost

```
Training data size: 5000
ADA BOOST
Training Classification Report 1 :
      precision    recall  f1-score   support

         0         0.84    0.78    0.81    2500
         1         0.80    0.85    0.82    2500

 accuracy          0.82    0.82    0.82    5000
 macro avg         0.82    0.82    0.82    5000
 weighted avg      0.82    0.82    0.82    5000

Test Classification Report 1 :
      precision    recall  f1-score   support

         0         0.82    0.75    0.79    12500
         1         0.77    0.83    0.80    12500

 accuracy          0.79    0.79    0.79    25000
 macro avg         0.80    0.79    0.79    25000
 weighted avg      0.80    0.79    0.79    25000
```

```
Training data size: 10000
ADA BOOST
Training Classification Report 2 :
      precision    recall  f1-score   support

         0         0.83    0.78    0.81    5000
         1         0.79    0.84    0.82    5000

 accuracy          0.81    0.81    0.81   10000
 macro avg         0.81    0.81    0.81   10000
 weighted avg      0.81    0.81    0.81   10000

Test Classification Report 2 :
      precision    recall  f1-score   support

         0         0.81    0.77    0.79    12500
         1         0.78    0.82    0.80    12500

 accuracy          0.80    0.80    0.80   25000
 macro avg         0.80    0.80    0.80   25000
 weighted avg      0.80    0.80    0.80   25000
```

```
Training data size: 15000
ADA BOOST
Training Classification Report 3 :
      precision    recall  f1-score   support

         0         0.83    0.77    0.80    7500
         1         0.79    0.84    0.81    7500

 accuracy          0.81    0.81    0.81   15000
 macro avg         0.81    0.81    0.81   15000
 weighted avg      0.81    0.81    0.81   15000

Test Classification Report 3 :
      precision    recall  f1-score   support

         0         0.82    0.77    0.79    12500
         1         0.78    0.83    0.81    12500

 accuracy          0.80    0.80    0.80   25000
 macro avg         0.80    0.80    0.80   25000
 weighted avg      0.80    0.80    0.80   25000
```

```
Training data size: 20000
ADA BOOST
Training Classification Report 4 :
      precision    recall  f1-score   support

         0         0.83    0.78    0.80   10000
         1         0.79    0.84    0.81   10000

 accuracy          0.81    0.81    0.81   20000
 macro avg         0.81    0.81    0.81   20000
 weighted avg      0.81    0.81    0.81   20000

Test Classification Report 4 :
      precision    recall  f1-score   support

         0         0.82    0.77    0.80    12500
         1         0.78    0.83    0.81    12500

 accuracy          0.80    0.80    0.80   25000
 macro avg         0.80    0.80    0.80   25000
 weighted avg      0.80    0.80    0.80   25000
```

```
Training data size: 25000
ADA BOOST
Training Classification Report 5 :
      precision    recall  f1-score   support

         0         0.83    0.77    0.80   12500
         1         0.79    0.84    0.81   12500

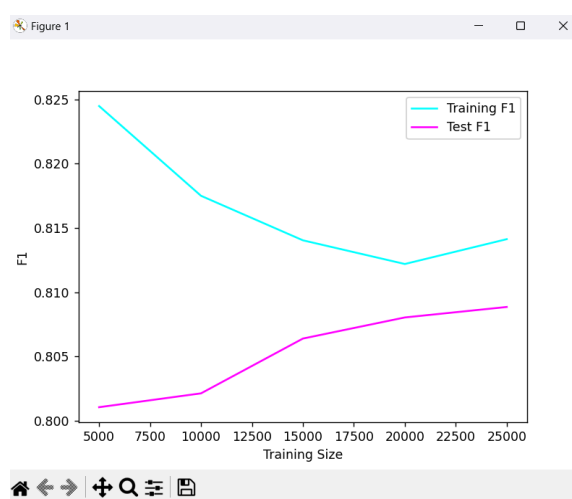
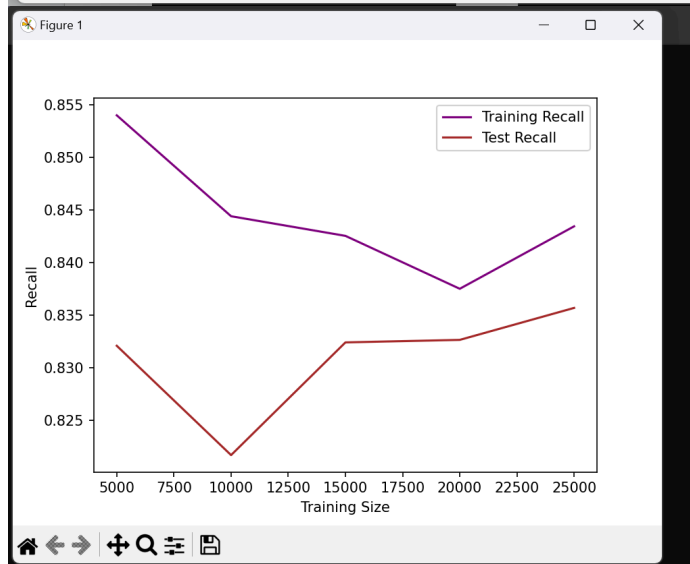
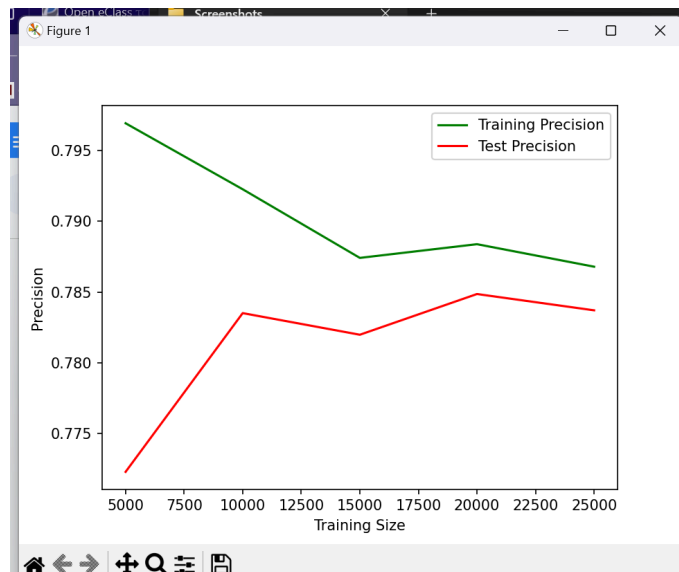
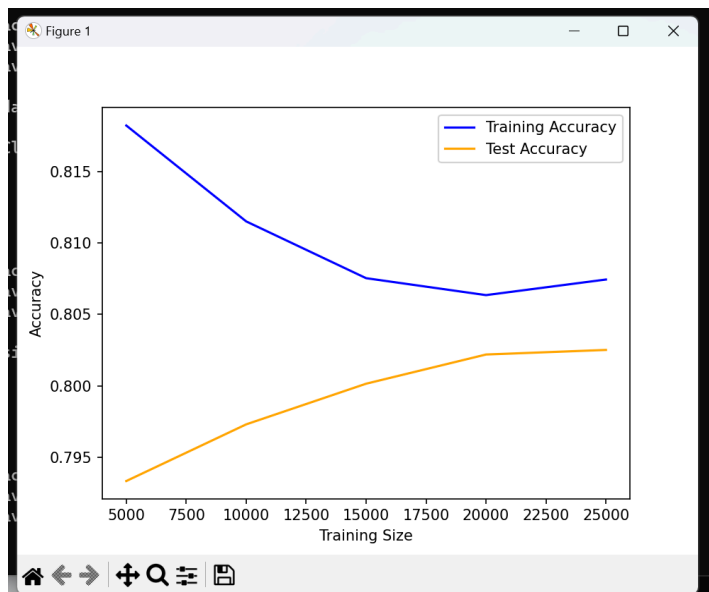
 accuracy          0.81    0.81    0.81   25000
 macro avg         0.81    0.81    0.81   25000
 weighted avg      0.81    0.81    0.81   25000

Test Classification Report 5 :
      precision    recall  f1-score   support

         0         0.82    0.77    0.80   12500
         1         0.78    0.84    0.81   12500

 accuracy          0.80    0.80    0.80   25000
 macro avg         0.80    0.80    0.80   25000
 weighted avg      0.80    0.80    0.80   25000

Execution_time: 622.9281389713287 seconds
```

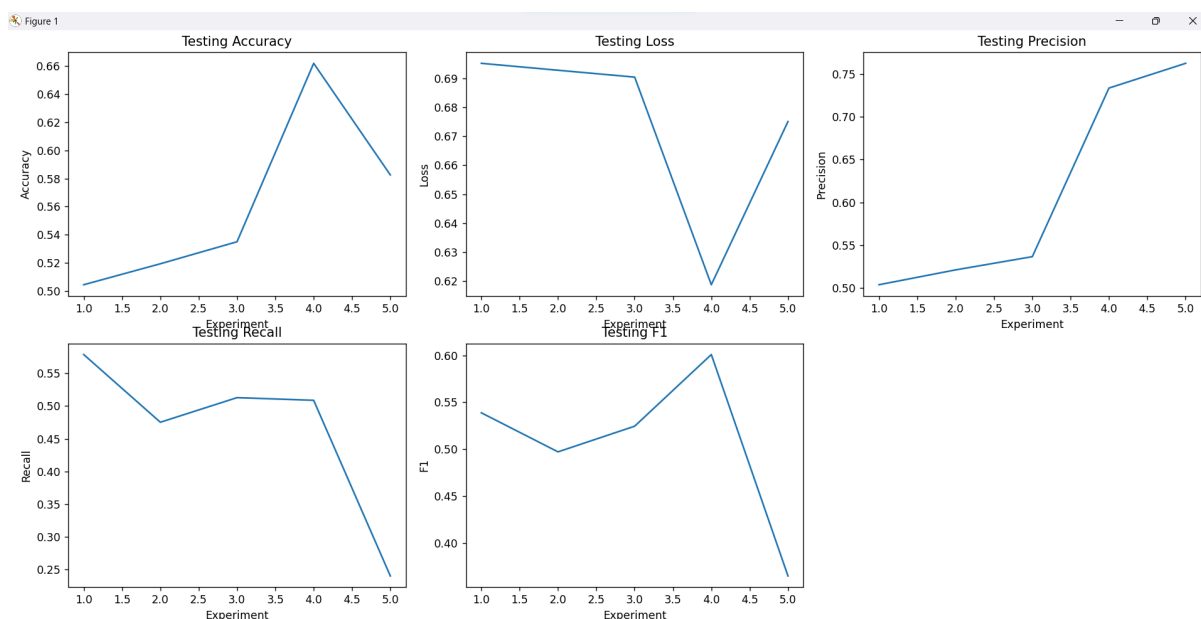
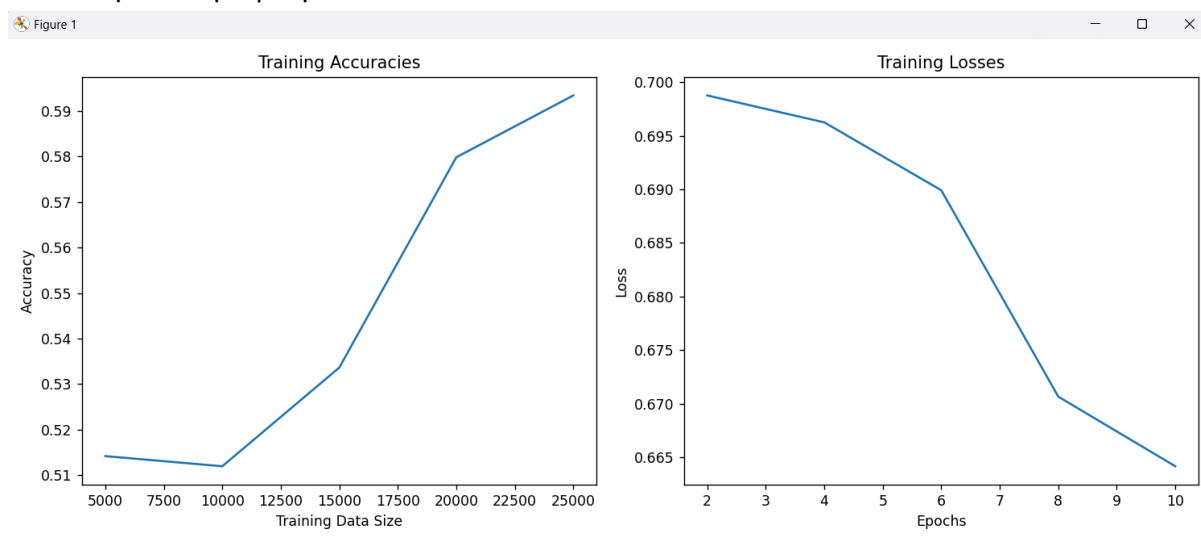


Οι βασικές διαφορές που παρατηρούμε σε σχέση με την δική μας υλοποίηση του αλγορίθμου AdaBoost είναι η πορεία των καμπυλών καθώς και η μη κοινή πορεία των δεδομένων εκπαίδευσης και ελέγχου. Στην έτοιμη υλοποίηση όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης: Το ποσοστό ορθότητας αυξάνεται στα test data (από 0.79 στο 0.80) ενώ μειώνεται στα train data (από 0.82 στο 0.81), η ίδια πορεία παρατηρείται και στα υπόλοιπα αποτελέσματα precision, recall, F1.

Ο χρόνος εκτέλεσης, τέλος, παρουσιάζει αρκετά μεγάλη διαφορά. Η έτοιμη υλοποίηση εκτελείται πολύ πιο γρήγορα και για τα 5 πειράματα.

Μέρος Γ

Με την χρήση RNN, που υλοποιούμε σε Tensorflow/Keras εξαγάγαμε τα παρακάτω αποτελέσματα-συμπεράσματα:



Η βασική διαφορά που παρατηρούμε σε σχέση με τις υλοποιήσεις των αλγορίθμων στα Μέρη Α και Β είναι οι τιμές των αποτελεσμάτων. Στην RNN υλοποίηση όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης: Στα test data το accuracy αυξάνεται και στο τέλος μειώνεται ελάχιστα, το precision έχει ανοδική πορεία, το recall και το F1 έχουν καθοδική πορεία ενώ το loss μειώνεται και στο τέλος αυξάνεται. Στα train data, το accuracy έχει σταθερή ανοδική πορεία ενώ το loss σταθερή καθοδική. Γενικά, τα αποτελέσματα είναι αρκετά χαμηλά σε σχέση με τις υπόλοιπες υλοποιήσεις.

```

1/1 [=====] - 0s 220ms/step
Epoch 1/2
WARNING:tensorflow:From C:\Users\mpele\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

157/157 [=====] - 17s 95ms/step - loss: 0.7012 - binary_accuracy: 0.5092
Epoch 2/2
157/157 [=====] - 13s 81ms/step - loss: 0.6963 - binary_accuracy: 0.5192
782/782 [=====] - 19s 24ms/step - loss: 0.6951 - binary_accuracy: 0.5044
Training Data Experiment No 1 for 2 epochs:
Accuracy(average): 0.51
Loss(average): 0.7
782/782 [=====] - 21s 27ms/step
Testing Data Experiment No 1 :
      precision    recall  f1-score   support

     0       0.51      0.43      0.46      12500
     1       0.50      0.58      0.54      12500

 accuracy
macro avg       0.50      0.50      0.50      25000
weighted avg       0.50      0.50      0.50      25000

```

```

100%|████████████████████████████████████████████████████████████████████████████████| 10000/10000 [00:00<00:00, 49111.33it/s]

Training data average document length = 241.3587
1/1 [=====] - 0s 141ms/step
Epoch 1/4
113/113 [=====] - 23s 68ms/step - loss: 0.7026 - binary_accuracy: 0.4950
Epoch 2/4
113/113 [=====] - 21s 67ms/step - loss: 0.6973 - binary_accuracy: 0.5100
Epoch 3/4
113/113 [=====] - 22s 69ms/step - loss: 0.6936 - binary_accuracy: 0.5188
Epoch 4/4
113/113 [=====] - 30s 97ms/step - loss: 0.6914 - binary_accuracy: 0.5241
782/782 [=====] - 19s 23ms/step - loss: 0.6927 - binary_accuracy: 0.5193
Training Data Experiment No 2 for 4 epochs:
Accuracy(average): 0.51
Loss(average): 0.7
782/782 [=====] - 13s 16ms/step
Testing Data Experiment No 2 :
      precision    recall  f1-score   support

     0       0.52      0.56      0.54      12500
     1       0.52      0.48      0.50      12500

 accuracy
macro avg       0.52      0.52      0.52      25000
weighted avg       0.52      0.52      0.52      25000

```

```

100%|████████████████████████████████████████████████████████████████████████████████| 15000/15000 [00:00<00:00, 68009.93it/s]

Training data average document length = 239.54093333333333
1/1 [=====] - 0s 105ms/step
Epoch 1/6
469/469 [=====] - 38s 77ms/step - loss: 0.6994 - binary_accuracy: 0.4989
Epoch 2/6
469/469 [=====] - 56s 118ms/step - loss: 0.6952 - binary_accuracy: 0.5127
Epoch 3/6
469/469 [=====] - 42s 90ms/step - loss: 0.6915 - binary_accuracy: 0.5313
Epoch 4/6
469/469 [=====] - 39s 83ms/step - loss: 0.6886 - binary_accuracy: 0.5370
Epoch 5/6
469/469 [=====] - 39s 83ms/step - loss: 0.6841 - binary_accuracy: 0.5558
Epoch 6/6
469/469 [=====] - 37s 80ms/step - loss: 0.6807 - binary_accuracy: 0.5663
782/782 [=====] - 15s 19ms/step - loss: 0.6904 - binary_accuracy: 0.5350
Training Data Experiment No 3 for 6 epochs:
Accuracy(average): 0.53
Loss(average): 0.69
782/782 [=====] - 20s 23ms/step
Testing Data Experiment No 3 :
      precision    recall  f1-score   support

     0       0.53      0.56      0.55      12500
     1       0.54      0.51      0.52      12500

 accuracy
macro avg       0.54      0.53      0.53      25000
weighted avg       0.54      0.54      0.53      25000

```

100%| 20000/20000 [00:00<00:00, 61209.63it/s]

Training data average document length = 238.86525

1/1 [=====] - 0s 147ms/step

Epoch 1/8

625/625 [=====] - 52s 80ms/step - loss: 0.6990 - binary_accuracy: 0.5031

Epoch 2/8

625/625 [=====] - 49s 78ms/step - loss: 0.6934 - binary_accuracy: 0.5263

Epoch 3/8

625/625 [=====] - 51s 81ms/step - loss: 0.6850 - binary_accuracy: 0.5510

Epoch 4/8

625/625 [=====] - 61s 98ms/step - loss: 0.6760 - binary_accuracy: 0.5803

Epoch 5/8

625/625 [=====] - 50s 79ms/step - loss: 0.6619 - binary_accuracy: 0.6087

Epoch 6/8

625/625 [=====] - 45s 73ms/step - loss: 0.6734 - binary_accuracy: 0.5854

Epoch 7/8

625/625 [=====] - 47s 75ms/step - loss: 0.6463 - binary_accuracy: 0.6309

Epoch 8/8

625/625 [=====] - 58s 92ms/step - loss: 0.6301 - binary_accuracy: 0.6528

782/782 [=====] - 18s 21ms/step - loss: 0.6188 - binary_accuracy: 0.6620

Training Data Experiment No 4 for 8 epochs:

Accuracy(average): 0.58

Loss(average): 0.67

782/782 [=====] - 25s 31ms/step

Testing Data Experiment No 4 :

	precision	recall	f1-score	support
0	0.62	0.82	0.71	12500
1	0.73	0.51	0.60	12500
accuracy			0.66	25000
macro avg	0.68	0.66	0.65	25000
weighted avg	0.68	0.66	0.65	25000

Training data average document length = 238.71488

1/1 [=====] - 0s 155ms/step

Epoch 1/10

782/782 [=====] - 72s 89ms/step - loss: 0.6982 - binary_accuracy: 0.5084

Epoch 2/10

782/782 [=====] - 63s 80ms/step - loss: 0.6900 - binary_accuracy: 0.5348

Epoch 3/10

782/782 [=====] - 66s 85ms/step - loss: 0.6807 - binary_accuracy: 0.5678

Epoch 4/10

782/782 [=====] - 85s 108ms/step - loss: 0.6686 - binary_accuracy: 0.5926

Epoch 5/10

782/782 [=====] - 72s 93ms/step - loss: 0.6646 - binary_accuracy: 0.6029

Epoch 6/10

782/782 [=====] - 68s 87ms/step - loss: 0.6619 - binary_accuracy: 0.6084

Epoch 7/10

782/782 [=====] - 81s 103ms/step - loss: 0.6398 - binary_accuracy: 0.6415

Epoch 8/10

782/782 [=====] - 62s 79ms/step - loss: 0.6508 - binary_accuracy: 0.6170

Epoch 9/10

782/782 [=====] - 60s 77ms/step - loss: 0.6511 - binary_accuracy: 0.6178

Epoch 10/10

782/782 [=====] - 70s 89ms/step - loss: 0.6359 - binary_accuracy: 0.6427

782/782 [=====] - 16s 19ms/step - loss: 0.6750 - binary_accuracy: 0.5826

Training Data Experiment No 5 for 10 epochs:

Accuracy(average): 0.59

Loss(average): 0.66

782/782 [=====] - 13s 17ms/step

Testing Data Experiment No 5 :

	precision	recall	f1-score	support
0	0.55	0.93	0.69	12500
1	0.76	0.24	0.36	12500
accuracy			0.58	25000
macro avg	0.66	0.58	0.53	25000
weighted avg	0.66	0.58	0.53	25000